WeIntern Data Science Internship — Week 1

Task 1: E-Commerce Dataset Cleaning

1. Objective

The objective of this task is to inspect and clean an e-commerce dataset by identifying missing values, duplicate records, data type issues, and invalid values.

2. Importing Libraries

In this step, I imported the pandas library to load, inspect, and clean the e-commerce dataset. Pandas provides functions for working with tabular data.

In [2]:
```
import pandas as pd

print("Pandas version:", pd.__version__)
```

```
Pandas version: 3.1.0
```

3. Loading the Dataset

I loaded the original CSV file from the `data/raw` folder into a pandas DataFrame. The original file will remain unchanged throughout the cleaning process.

In [4]:
```
df = pd.read_csv("../data/raw/Ecommerce_2000.csv")

print("Dataset loaded successfully!")

display(df.head())
```

```
Dataset loaded successfully!
```

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	Orde
0	ORD-010001	CUST-05677	Kimberly	Davis	MT	Fairview	5/22/2022	
1	ORD-010002	CUST-07683	Linda	Garcia	AZ	Georgetown	10/17/2024	
2	ORD-010003	CUST-05017	Steven	Baker	VA	Clinton	6/8/2024	
3	ORD-010004	CUST-04913	Laura	Thomas	MO	Salem	10/24/2024	
4	ORD-010005	CUST-04532	Thomas	Turner	AZ	Springfield	7/22/2023	

5 rows × 23 columns

4. Raw Dataset Inspection

In this step, I inspected the dataset's dimensions, column names, data types, and sample records before performing any cleaning.

In [5]:
```
# Dataset dimensions
print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

# Column names
print("\nColumn names:")
print(df.columns.tolist())

# Data types and non-null counts
print("\nDataset information:")
df.info()

# Sample records
print("\nFirst five records:")
display(df.head())
```

```
Number of rows: 2000
Number of columns: 23

Column names:
['OrderID', 'CustomerID', 'FirstName', 'LastName', 'State', 'City', 'Order
Date', 'OrderSeason', 'OrderDayOfWeek', 'ProductCategory', 'ProductSubCate
gory', 'UnitPrice', 'Quantity', 'TotalAmount', 'DiscountPercent', 'Discoun
tAmount', 'FinalAmount', 'PaymentMethod', 'DeviceType', 'MembershipTier',
'ShippingDays', 'Returned', 'ReviewScore']

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 23 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   OrderID             2000 non-null   str
 1   CustomerID          2000 non-null   str
 2   FirstName           2000 non-null   str
 3   LastName            2000 non-null   str
 4   State               2000 non-null   str
 5   City                2000 non-null   str
 6   OrderDate           2000 non-null   str
 7   OrderSeason         2000 non-null   str
 8   OrderDayOfWeek      2000 non-null   str
 9   ProductCategory     2000 non-null   str
 10  ProductSubCategory  2000 non-null   str
 11  UnitPrice           2000 non-null   float64
 12  Quantity            2000 non-null   int64
 13  TotalAmount         2000 non-null   float64
 14  DiscountPercent     2000 non-null   int64
 15  DiscountAmount      2000 non-null   float64
 16  FinalAmount         2000 non-null   float64
 17  PaymentMethod       2000 non-null   str
 18  DeviceType          2000 non-null   str
 19  MembershipTier      2000 non-null   str
 20  ShippingDays        2000 non-null   int64
 21  Returned            2000 non-null   str
 22  ReviewScore         2000 non-null   int64
dtypes: float64(4), int64(4), str(15)
memory usage: 359.5 KB

First five records:
```

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	Orde
0	ORD-010001	CUST-05677	Kimberly	Davis	MT	Fairview	5/22/2022	
1	ORD-010002	CUST-07683	Linda	Garcia	AZ	Georgetown	10/17/2024	
2	ORD-010003	CUST-05017	Steven	Baker	VA	Clinton	6/8/2024	
3	ORD-010004	CUST-04913	Laura	Thomas	MO	Salem	10/24/2024	
4	ORD-010005	CUST-04532	Thomas	Turner	AZ	Springfield	7/22/2023	

5 rows × 23 columns

Initial Inspection Findings

- The raw dataset contains 2,000 rows and 23 columns.
- The dataset includes order, customer, location, product, pricing, payment, shipping, return, and review information.
- The columns contain text and numeric data types.
- The `OrderDate` column is currently stored as text and requires date validation.
- No missing values were indicated in the displayed dataset information.
- The original dataset has not been modified during this inspection.

5. Missing Value Analysis

In [6]:
```
missing_report = pd.DataFrame({
    "Missing Count": df.isnull().sum(),
    "Missing Percentage (%)": (
        df.isnull().sum() / len(df) * 100
    ).round(2)
})

display(missing_report)

print(
    "Total missing-value cells:",
    int(df.isnull().sum().sum())
)
```

	Missing Count	Missing Percentage (%)
OrderID	0	0.0
CustomerID	0	0.0
FirstName	0	0.0
LastName	0	0.0
State	0	0.0
City	0	0.0
OrderDate	0	0.0
OrderSeason	0	0.0
OrderDayOfWeek	0	0.0
ProductCategory	0	0.0
ProductSubCategory	0	0.0
UnitPrice	0	0.0
Quantity	0	0.0
TotalAmount	0	0.0
DiscountPercent	0	0.0
DiscountAmount	0	0.0
FinalAmount	0	0.0
PaymentMethod	0	0.0
DeviceType	0	0.0
MembershipTier	0	0.0
ShippingDays	0	0.0
Returned	0	0.0
ReviewScore	0	0.0

```
Total missing-value cells: 0
```

Missing Value Analysis — Findings

The missing value analysis was performed on all 23 columns of the dataset.

- Total records examined: 2,000
- Total columns examined: 23
- Total missing-value cells: 0
- Missing-value percentage: 0% across all columns

Cleaning decision: No missing-value imputation or row removal was required because the dataset contains no missing values. Therefore, no changes were made

to the dataset for missing-value handling.

6. Duplicate Record Analysis

Duplicate analysis is performed to identify fully duplicated records and repeated order identifiers.

Fully duplicated rows may indicate accidental copies, while repeated OrderID values require further investigation to determine whether they represent duplicate orders or valid repeated entries.

Only confirmed duplicate records will be removed.

In [7]:
```
# Check fully duplicated rows
exact_duplicate_count = df.duplicated().sum()

print("Exact duplicate rows:", exact_duplicate_count)

# Check repeated OrderID values
repeated_order_id_rows = df[
    df.duplicated(subset="OrderID", keep=False)
].sort_values("OrderID")

print("Rows belonging to repeated OrderIDs:",
      len(repeated_order_id_rows))

print("Number of distinct repeated OrderIDs:",
      repeated_order_id_rows["OrderID"].nunique())

display(repeated_order_id_rows.head(10))
```

```
Exact duplicate rows: 20
Rows belonging to repeated OrderIDs: 39
Number of distinct repeated OrderIDs: 19
```

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	Or
1946	ORD-010190-X	CUST-06882	Ronald	Perez	OK	Bristol	6/6/2022	
1784	ORD-010190-X	CUST-06882	Ronald	Perez	OK	Bristol	6/6/2022	
1918	ORD-010231-X	CUST-05597	Jessica	Thomas	AL	Salem	12/23/2024	
1873	ORD-010231-X	CUST-05597	Jessica	Thomas	AL	Salem	12/23/2024	
1897	ORD-010503-X	CUST-07597	Steven	Nelson	VA	Springfield	9/24/2024	
1883	ORD-010503-X	CUST-07597	Steven	Nelson	VA	Springfield	9/24/2024	
1913	ORD-010564-X	CUST-07516	Daniel	Phillips	NV	Salem	11/23/2024	
1975	ORD-010564-X	CUST-07516	Daniel	Phillips	NV	Salem	11/23/2024	
1888	ORD-010684-X	CUST-01671	Kenneth	Martinez	NC	Durham	2/28/2024	
1964	ORD-010684-X	CUST-01671	Kenneth	Martinez	NC	Durham	2/28/2024	

10 rows × 23 columns

Duplicate Analysis — Initial Findings

The duplicate analysis identified 20 fully duplicated rows. A separate check of the `OrderID` column found 39 rows associated with 19 distinct repeated order IDs.

These results indicate that exact duplicate rows and repeated order identifiers must be investigated separately. Repeated order IDs will not automatically be treated as errors because they may represent valid transactions or repeated entries.

Cleaning decision: Further inspection is required before deciding how to handle the repeated order IDs. No records have been removed at this stage.

In [8]:

```
# Display all rows associated with repeated OrderIDs
display(
    df[df.duplicated(subset="OrderID", keep=False)]
    .sort_values("OrderID")
)
```

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	C
1946	ORD-010190-X	CUST-06882	Ronald	Perez	OK	Bristol	6/6/2022	
1784	ORD-010190-X	CUST-06882	Ronald	Perez	OK	Bristol	6/6/2022	
1918	ORD-010231-X	CUST-05597	Jessica	Thomas	AL	Salem	12/23/2024	
1873	ORD-010231-X	CUST-05597	Jessica	Thomas	AL	Salem	12/23/2024	
1897	ORD-010503-X	CUST-07597	Steven	Nelson	VA	Springfield	9/24/2024	
1883	ORD-010503-X	CUST-07597	Steven	Nelson	VA	Springfield	9/24/2024	
1913	ORD-010564-X	CUST-07516	Daniel	Phillips	NV	Salem	11/23/2024	
1975	ORD-010564-X	CUST-07516	Daniel	Phillips	NV	Salem	11/23/2024	
1888	ORD-010684-X	CUST-01671	Kenneth	Martinez	NC	Durham	2/28/2024	
1964	ORD-010684-X	CUST-01671	Kenneth	Martinez	NC	Durham	2/28/2024	
1999	ORD-010730-X	CUST-07756	Robert	Turner	ME	Springfield	11/26/2022	
1830	ORD-010730-X	CUST-07756	Robert	Turner	ME	Springfield	11/26/2022	
1812	ORD-010956-X	CUST-03448	Matthew	Thompson	AR	Clinton	4/22/2024	
1767	ORD-010956-X	CUST-03448	Matthew	Thompson	AR	Clinton	4/22/2024	
1787	ORD-010958-	CUST-03239	Michael	Jones	NH	Bristol	5/30/2022	

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	C
	X							
1785	ORD-010958-X	CUST-03239	Michael	Jones	NH	Bristol	5/30/2022	
1876	ORD-011052-X	CUST-01327	Timothy	Miller	SC	Fairview	6/30/2023	
1936	ORD-011052-X	CUST-01327	Timothy	Miller	SC	Fairview	6/30/2023	
1847	ORD-011077-X	CUST-05321	Ruth	Young	NY	New York City	1/2/2023	
1997	ORD-011077-X	CUST-05321	Ruth	Young	NY	New York City	1/2/2023	
1984	ORD-011105-X	CUST-06871	Sharon	Wilson	MS	Fairview	11/20/2023	
1819	ORD-011105-X	CUST-06871	Sharon	Wilson	MS	Fairview	11/20/2023	
1980	ORD-011137-X	CUST-08831	Susan	Clark	CT	Georgetown	10/31/2022	
1972	ORD-011137-X	CUST-08831	Susan	Clark	CT	Georgetown	10/31/2022	
1947	ORD-011249-X	CUST-07323	Anthony	Nelson	NV	Greenville	9/12/2023	
1992	ORD-011249-X	CUST-07323	Anthony	Nelson	NV	Greenville	9/12/2023	
1962	ORD-011249-X	CUST-07323	Anthony	Nelson	NV	Greenville	9/12/2023	
1741	ORD-011295-X	CUST-06888	Kenneth	Harris	AL	Greenville	10/19/2022	
1837	ORD-011295-X	CUST-06888	Kenneth	Harris	AL	Greenville	10/19/2022	

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	C
1809	ORD-011364-X	CUST-08409	Joshua	Miller	ME	Fairview	1/13/2023	
1939	ORD-011364-X	CUST-08409	Joshua	Miller	ME	Fairview	1/13/2023	
1839	ORD-011486-X	CUST-01262	Michelle	Baker	NE	Salem	5/18/2024	
1893	ORD-011486-X	CUST-01262	Michelle	Baker	NE	Salem	5/18/2024	
1969	ORD-011496-X	CUST-01844	Lisa	Smith	OH	Columbus	7/28/2024	
1862	ORD-011496-X	CUST-01844	Lisa	Smith	OH	Columbus	7/28/2024	
1938	ORD-011580-X	CUST-07076	Edward	King	SD	Clinton	8/11/2022	
1831	ORD-011580-X	CUST-07076	Edward	King	SD	Clinton	8/11/2022	
1932	ORD-011598-X	CUST-02113	Mark	Wright	MT	Bristol	10/25/2022	
1885	ORD-011598-X	CUST-02113	Mark	Wright	MT	Bristol	10/25/2022	

39 rows × 23 columns

In [9]:
```
order_id_counts = df["OrderID"].value_counts()

repeated_ids = order_id_counts[order_id_counts > 1]

display(
    repeated_ids.rename("Occurrence Count")
    .reset_index()
    .rename(columns={"index": "OrderID"})
)
```

	OrderID	Occurrence Count
0	ORD-011249-X	3
1	ORD-011295-X	2
2	ORD-010956-X	2
3	ORD-010190-X	2
4	ORD-010958-X	2
5	ORD-011364-X	2
6	ORD-011105-X	2
7	ORD-010730-X	2
8	ORD-011580-X	2
9	ORD-011486-X	2
10	ORD-011077-X	2
11	ORD-011496-X	2
12	ORD-010231-X	2
13	ORD-011052-X	2
14	ORD-010503-X	2
15	ORD-011598-X	2
16	ORD-010684-X	2
17	ORD-010564-X	2
18	ORD-011137-X	2

In [10]:
```
# Display all rows for repeated OrderIDs
repeated_orders = df[
    df.duplicated(subset="OrderID", keep=False)
].sort_values("OrderID")

display(repeated_orders)
```

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	C
1946	ORD-010190-X	CUST-06882	Ronald	Perez	OK	Bristol	6/6/2022	
1784	ORD-010190-X	CUST-06882	Ronald	Perez	OK	Bristol	6/6/2022	
1918	ORD-010231-X	CUST-05597	Jessica	Thomas	AL	Salem	12/23/2024	
1873	ORD-010231-X	CUST-05597	Jessica	Thomas	AL	Salem	12/23/2024	
1897	ORD-010503-X	CUST-07597	Steven	Nelson	VA	Springfield	9/24/2024	
1883	ORD-010503-X	CUST-07597	Steven	Nelson	VA	Springfield	9/24/2024	
1913	ORD-010564-X	CUST-07516	Daniel	Phillips	NV	Salem	11/23/2024	
1975	ORD-010564-X	CUST-07516	Daniel	Phillips	NV	Salem	11/23/2024	
1888	ORD-010684-X	CUST-01671	Kenneth	Martinez	NC	Durham	2/28/2024	
1964	ORD-010684-X	CUST-01671	Kenneth	Martinez	NC	Durham	2/28/2024	
1999	ORD-010730-X	CUST-07756	Robert	Turner	ME	Springfield	11/26/2022	
1830	ORD-010730-X	CUST-07756	Robert	Turner	ME	Springfield	11/26/2022	
1812	ORD-010956-X	CUST-03448	Matthew	Thompson	AR	Clinton	4/22/2024	
1767	ORD-010956-X	CUST-03448	Matthew	Thompson	AR	Clinton	4/22/2024	
1787	ORD-010958-	CUST-03239	Michael	Jones	NH	Bristol	5/30/2022	

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	C
	X							
1785	ORD-010958-X	CUST-03239	Michael	Jones	NH	Bristol	5/30/2022	
1876	ORD-011052-X	CUST-01327	Timothy	Miller	SC	Fairview	6/30/2023	
1936	ORD-011052-X	CUST-01327	Timothy	Miller	SC	Fairview	6/30/2023	
1847	ORD-011077-X	CUST-05321	Ruth	Young	NY	New York City	1/2/2023	
1997	ORD-011077-X	CUST-05321	Ruth	Young	NY	New York City	1/2/2023	
1984	ORD-011105-X	CUST-06871	Sharon	Wilson	MS	Fairview	11/20/2023	
1819	ORD-011105-X	CUST-06871	Sharon	Wilson	MS	Fairview	11/20/2023	
1980	ORD-011137-X	CUST-08831	Susan	Clark	CT	Georgetown	10/31/2022	
1972	ORD-011137-X	CUST-08831	Susan	Clark	CT	Georgetown	10/31/2022	
1947	ORD-011249-X	CUST-07323	Anthony	Nelson	NV	Greenville	9/12/2023	
1992	ORD-011249-X	CUST-07323	Anthony	Nelson	NV	Greenville	9/12/2023	
1962	ORD-011249-X	CUST-07323	Anthony	Nelson	NV	Greenville	9/12/2023	
1741	ORD-011295-X	CUST-06888	Kenneth	Harris	AL	Greenville	10/19/2022	
1837	ORD-011295-X	CUST-06888	Kenneth	Harris	AL	Greenville	10/19/2022	

	OrderID	CustomerID	FirstName	LastName	State	City	OrderDate	C
1809	ORD-011364-X	CUST-08409	Joshua	Miller	ME	Fairview	1/13/2023	
1939	ORD-011364-X	CUST-08409	Joshua	Miller	ME	Fairview	1/13/2023	
1839	ORD-011486-X	CUST-01262	Michelle	Baker	NE	Salem	5/18/2024	
1893	ORD-011486-X	CUST-01262	Michelle	Baker	NE	Salem	5/18/2024	
1969	ORD-011496-X	CUST-01844	Lisa	Smith	OH	Columbus	7/28/2024	
1862	ORD-011496-X	CUST-01844	Lisa	Smith	OH	Columbus	7/28/2024	
1938	ORD-011580-X	CUST-07076	Edward	King	SD	Clinton	8/11/2022	
1831	ORD-011580-X	CUST-07076	Edward	King	SD	Clinton	8/11/2022	
1932	ORD-011598-X	CUST-02113	Mark	Wright	MT	Bristol	10/25/2022	
1885	ORD-011598-X	CUST-02113	Mark	Wright	MT	Bristol	10/25/2022	

39 rows × 23 columns

In [11]:
```
# Compare important fields for repeated OrderIDs
columns_to_check = [
    "OrderID",
    "CustomerID",
    "OrderDate",
    "ProductCategory",
    "ProductSubCategory",
    "UnitPrice",
    "Quantity",
    "TotalAmount",
    "FinalAmount"
]

display(repeated_orders[columns_to_check])
```

	OrderID	CustomerID	OrderDate	ProductCategory	ProductSubCategory	UnitI
1946	ORD-010190-X	CUST-06882	6/6/2022	Books	Self-Help	4
1784	ORD-010190-X	CUST-06882	6/6/2022	Books	Self-Help	4
1918	ORD-010231-X	CUST-05597	12/23/2024	Grocery	Snacks	
1873	ORD-010231-X	CUST-05597	12/23/2024	Grocery	Snacks	
1897	ORD-010503-X	CUST-07597	9/24/2024	Beauty & Personal Care	Haircare	5
1883	ORD-010503-X	CUST-07597	9/24/2024	Beauty & Personal Care	Haircare	5
1913	ORD-010564-X	CUST-07516	11/23/2024	Grocery	Snacks	3
1975	ORD-010564-X	CUST-07516	11/23/2024	Grocery	Snacks	3
1888	ORD-010684-X	CUST-01671	2/28/2024	Sports & Outdoors	Cycling	15
1964	ORD-010684-X	CUST-01671	2/28/2024	Sports & Outdoors	Cycling	15
1999	ORD-010730-X	CUST-07756	11/26/2022	Sports & Outdoors	Water Sports	38
1830	ORD-010730-X	CUST-07756	11/26/2022	Sports & Outdoors	Water Sports	38
1812	ORD-010956-X	CUST-03448	4/22/2024	Automotive	Parts	15
1767	ORD-010956-X	CUST-03448	4/22/2024	Automotive	Parts	15
1787	ORD-010958-	CUST-03239	5/30/2022	Health & Wellness	Fitness Accessories	10

	OrderID	CustomerID	OrderDate	ProductCategory	ProductSubCategory	UnitI
	X					
1785	ORD-010958-X	CUST-03239	5/30/2022	Health & Wellness	Fitness Accessories	10
1876	ORD-011052-X	CUST-01327	6/30/2023	Books	History	1
1936	ORD-011052-X	CUST-01327	6/30/2023	Books	History	1
1847	ORD-011077-X	CUST-05321	1/2/2023	Grocery	Dairy	2
1997	ORD-011077-X	CUST-05321	1/2/2023	Grocery	Dairy	2
1984	ORD-011105-X	CUST-06871	11/20/2023	Electronics	Headphones	105
1819	ORD-011105-X	CUST-06871	11/20/2023	Electronics	Headphones	105
1980	ORD-011137-X	CUST-08831	10/31/2022	Clothing	Bottoms	19
1972	ORD-011137-X	CUST-08831	10/31/2022	Clothing	Bottoms	19
1947	ORD-011249-X	CUST-07323	9/12/2023	Clothing	Accessories	15
1992	ORD-011249-X	CUST-07323	9/12/2023	Clothing	Accessories	15
1962	ORD-011249-X	CUST-07323	9/12/2023	Clothing	Accessories	15
1741	ORD-011295-X	CUST-06888	10/19/2022	Clothing	Footwear	8
1837	ORD-011295-X	CUST-06888	10/19/2022	Clothing	Footwear	8

	OrderID	CustomerID	OrderDate	ProductCategory	ProductSubCategory	UnitI
1809	ORD-011364-X	CUST-08409	1/13/2023	Grocery	Beverages	2
1939	ORD-011364-X	CUST-08409	1/13/2023	Grocery	Beverages	2
1839	ORD-011486-X	CUST-01262	5/18/2024	Beauty & Personal Care	Fragrances	4
1893	ORD-011486-X	CUST-01262	5/18/2024	Beauty & Personal Care	Fragrances	4
1969	ORD-011496-X	CUST-01844	7/28/2024	Sports & Outdoors	Cycling	7
1862	ORD-011496-X	CUST-01844	7/28/2024	Sports & Outdoors	Cycling	7
1938	ORD-011580-X	CUST-07076	8/11/2022	Health & Wellness	Mental Wellness	9
1831	ORD-011580-X	CUST-07076	8/11/2022	Health & Wellness	Mental Wellness	9
1932	ORD-011598-X	CUST-02113	10/25/2022	Beauty & Personal Care	Skincare	2
1885	ORD-011598-X	CUST-02113	10/25/2022	Beauty & Personal Care	Skincare	2

In [13]:
```
# Check how many unique full records exist for each repeated OrderID
duplicate_check = (
    repeated_orders
    .drop_duplicates()
    .groupby("OrderID")
    .size()
)

print("Number of unique full records for each repeated OrderID:")
display(duplicate_check)

print(
    "Repeated OrderIDs with identical records:",
    (duplicate_check == 1).sum()
)

print(
```

```
    "Repeated OrderIDs with different records:",
    (duplicate_check > 1).sum()
)
```

```
Number of unique full records for each repeated OrderID:
OrderID
ORD-010190-X    1
ORD-010231-X    1
ORD-010503-X    1
ORD-010564-X    1
ORD-010684-X    1
ORD-010730-X    1
ORD-010956-X    1
ORD-010958-X    1
ORD-011052-X    1
ORD-011077-X    1
ORD-011105-X    1
ORD-011137-X    1
ORD-011249-X    1
ORD-011295-X    1
ORD-011364-X    1
ORD-011486-X    1
ORD-011496-X    1
ORD-011580-X    1
ORD-011598-X    1
dtype: int64
Repeated OrderIDs with identical records: 19
Repeated OrderIDs with different records: 0
```

Duplicate Analysis — Final Findings

The duplicate analysis identified 20 fully duplicated rows in the original dataset. Additionally, 39 rows were associated with 19 distinct repeated `OrderID` values.

After removing exact duplicates for investigation, each of the 19 repeated order IDs had exactly one unique full record. No repeated order ID had different record versions.

Cleaning decision: The repeated entries are treated as duplicate copies because their full records are identical. Exact duplicate rows will be removed from the cleaned dataset while retaining one copy of each record. The raw dataset will remain unchanged.

7. Removing Exact Duplicate Rows

The investigation identified 20 exact duplicate rows in the original dataset. The repeated `OrderID` groups were also examined, and all 19 groups contained identical full records.

To prevent duplicate records from affecting future analysis, a separate copy of the original dataset will be created. Extra copies of exact duplicate rows will be removed, while one copy of each record will be retained. The original dataset will remain unchanged.

In [14]:
```python
# Create a separate copy of the original dataset
df_clean = df.copy()

# Remove exact duplicate rows from the copy
df_clean = df_clean.drop_duplicates().copy()

# Check the results
print("Original dataset rows:", len(df))
print("Cleaned dataset rows:", len(df_clean))
print("Duplicate rows remaining:", df_clean.duplicated().sum())
```

```
Original dataset rows: 2000
Cleaned dataset rows: 1980
Duplicate rows remaining: 0
```

Duplicate Removal — Results

The original dataset contained 2,000 rows, including 20 extra copies of exact duplicate records. After applying `drop_duplicates()` to a separate copy of the dataset, the cleaned DataFrame contains 1,980 rows.

A verification check confirmed that zero exact duplicate rows remain in `df_clean`.

The original DataFrame, `df`, was preserved without modification. The cleaned DataFrame, `df_clean`, will be used for the remaining validation and cleaning steps.

8. Data Validation

Data validation checks whether values follow reasonable rules and expected formats. In this step, we will examine dates, quantities, prices, discounts, transaction amounts, shipping days, review scores, and categorical values.

The checks will help us identify potential data-quality issues before deciding whether any additional cleaning is necessary.

In [15]:
```python
# Check data types and basic statistics
print("Data types and non-null counts:")
df_clean.info()

print("\nNumeric column summary:")
display(df_clean.describe())

print("\nUnique values in selected categorical columns:")
for column in ["Returned", "OrderSeason", "OrderDayOfWeek",
               "PaymentMethod", "DeviceType", "MembershipTier"]:
    print(f"\n{column}:")
    print(df_clean[column].value_counts(dropna=False))
```

```
Data types and non-null counts:
<class 'pandas.DataFrame'>
Index: 1980 entries, 0 to 1998
Data columns (total 23 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   OrderID             1980 non-null   str
 1   CustomerID          1980 non-null   str
 2   FirstName           1980 non-null   str
 3   LastName            1980 non-null   str
 4   State               1980 non-null   str
 5   City                1980 non-null   str
 6   OrderDate           1980 non-null   str
 7   OrderSeason         1980 non-null   str
 8   OrderDayOfWeek      1980 non-null   str
 9   ProductCategory     1980 non-null   str
 10  ProductSubCategory  1980 non-null   str
 11  UnitPrice           1980 non-null   float64
 12  Quantity            1980 non-null   int64
 13  TotalAmount         1980 non-null   float64
 14  DiscountPercent     1980 non-null   int64
 15  DiscountAmount      1980 non-null   float64
 16  FinalAmount         1980 non-null   float64
 17  PaymentMethod       1980 non-null   str
 18  DeviceType          1980 non-null   str
 19  MembershipTier      1980 non-null   str
 20  ShippingDays        1980 non-null   int64
 21  Returned            1980 non-null   str
 22  ReviewScore         1980 non-null   int64
dtypes: float64(4), int64(4), str(15)
memory usage: 371.2 KB

Numeric column summary:
```

	UnitPrice	Quantity	TotalAmount	DiscountPercent	DiscountAmount	Fi
count	1980.000000	1980.000000	1980.000000	1980.000000	1980.000000	1
mean	162.321843	1.477273	242.720737	5.141414	13.277424	
std	229.156978	0.867112	422.010277	7.399671	46.321996	
min	2.050000	1.000000	2.390000	0.000000	0.000000	
25%	36.182500	1.000000	42.562500	0.000000	0.000000	
50%	81.485000	1.000000	108.880000	0.000000	0.000000	
75%	173.995000	2.000000	256.027500	10.000000	7.280000	
max	1283.780000	5.000000	5994.200000	25.000000	1066.240000	5

```
Unique values in selected categorical columns:

Returned:
Returned
No     1854
Yes     126
Name: count, dtype: int64

OrderSeason:
OrderSeason
Winter    544
Spring    487
Fall      481
Summer    468
Name: count, dtype: int64

OrderDayOfWeek:
OrderDayOfWeek
Tuesday      314
Thursday     303
Monday       301
Sunday       274
Wednesday    272
Saturday     261
Friday       255
Name: count, dtype: int64

PaymentMethod:
PaymentMethod
Buy Now Pay Later    319
Gift Card            310
Apple Pay            296
Debit Card           267
PayPal               267
Credit Card          264
Google Pay           257
Name: count, dtype: int64

DeviceType:
DeviceType
Desktop    682
Mobile     650
Tablet     648
Name: count, dtype: int64

MembershipTier:
MembershipTier
Silver      515
Bronze      502
Gold        484
Platinum    479
Name: count, dtype: int64
```

8.1 Order Date Validation

The `OrderDate` column is currently stored as a string (`str`) rather than a date type. Date validation will check whether the values can be interpreted as valid dates.

The conversion will first be tested on a separate Series so that invalid date values can be identified before modifying the cleaned dataset.

In [16]:
```
# Test whether OrderDate values can be parsed as dates
parsed_dates = pd.to_datetime(
    df_clean["OrderDate"],
    format="%m/%d/%Y",
    errors="coerce"
)

# Count dates that could not be parsed
invalid_date_count = parsed_dates.isna().sum()

print("Invalid dates:", invalid_date_count)

if invalid_date_count == 0:
    print("Earliest order date:", parsed_dates.min())
    print("Latest order date:", parsed_dates.max())
else:
    print("Rows with invalid dates:")
    display(df_clean.loc[parsed_dates.isna(), ["OrderID", "OrderDate"]])
```

```
Invalid dates: 0
Earliest order date: 2022-01-01 00:00:00
Latest order date: 2025-02-28 00:00:00
```

Order Date Validation — Findings

The `OrderDate` column was tested using the date format `%m/%d/%Y`.

- Invalid dates found: 0
- Earliest order date: January 1, 2022
- Latest order date: February 28, 2025

Cleaning decision: All order dates were successfully parsed. No records need to be removed because of invalid dates. The column will be converted to a proper datetime data type for easier sorting and date-based analysis.

8.2 Transaction Amount Validation

The transaction amount columns will be checked to determine whether the recorded totals and discounts are consistent with the unit price, quantity, and discount percentage.

For each record, the expected total amount is calculated as `UnitPrice × Quantity`. The expected discount is calculated from the total amount and discount percentage, and the expected final amount is calculated by subtracting the discount amount from the total amount.

Any differences will be investigated before deciding whether corrections are necessary.

In [17]:
```python
# Calculate expected transaction amounts
expected_total = df_clean["UnitPrice"] * df_clean["Quantity"]

expected_discount = (
    expected_total * df_clean["DiscountPercent"] / 100
).round(2)

expected_final = expected_total - expected_discount

# Compare calculated amounts with recorded amounts
total_mismatch = ~(
    (df_clean["TotalAmount"] - expected_total).abs() < 0.011
)

discount_mismatch = ~(
    (df_clean["DiscountAmount"] - expected_discount).abs() < 0.011
)

final_mismatch = ~(
    (df_clean["FinalAmount"] - expected_final).abs() < 0.011
)

print("TotalAmount mismatches:", total_mismatch.sum())
print("DiscountAmount mismatches:", discount_mismatch.sum())
print("FinalAmount mismatches:", final_mismatch.sum())
```

```
TotalAmount mismatches: 0
DiscountAmount mismatches: 0
FinalAmount mismatches: 0
```

Transaction Amount Validation — Findings

The transaction amount validation was performed on all 1,980 rows in the cleaned DataFrame.

- `TotalAmount` mismatches: 0
- `DiscountAmount` mismatches: 0
- `FinalAmount` mismatches: 0

The recorded total amounts matched `UnitPrice × Quantity`. The recorded discount amounts matched the calculated discounts, and the final amounts matched the total amounts minus the discounts, allowing for rounding.

Cleaning decision: No corrections were required for these three transaction amount columns because all records passed the tested calculations.

8.3 Numeric Value Validation

The numeric columns will be checked for values that violate basic data-quality rules. Quantities should be positive, unit prices should be greater than zero, discount percentages should be between 0 and 100, shipping days should not be negative, and review scores should be within the expected range of 1 to 5.

The results will determine whether any numeric values require further investigation.

In [18]:
```
# Check for invalid numeric values
print("Quantity <= 0:", (df_clean["Quantity"] <= 0).sum())
print("UnitPrice <= 0:", (df_clean["UnitPrice"] <= 0).sum())
print(
    "DiscountPercent outside 0–100:",
    (
        (df_clean["DiscountPercent"] < 0) |
        (df_clean["DiscountPercent"] > 100)
    ).sum()
)
print("ShippingDays < 0:", (df_clean["ShippingDays"] < 0).sum())
print(
    "ReviewScore outside 1–5:",
    (
        (df_clean["ReviewScore"] < 1) |
        (df_clean["ReviewScore"] > 5)
    ).sum()
)
```

```
Quantity <= 0: 0
UnitPrice <= 0: 0
DiscountPercent outside 0–100: 0
ShippingDays < 0: 0
ReviewScore outside 1–5: 0
```

Numeric Value Validation — Findings

The numeric validation checks were performed on all 1,980 records in the cleaned DataFrame.

- Invalid quantities (less than or equal to zero): 0
- Invalid unit prices (less than or equal to zero): 0
- Discount percentages outside the 0–100 range: 0
- Negative shipping durations: 0
- Review scores outside the 1–5 range: 0

Cleaning decision: No corrections were required for these numeric columns because all records passed the specified validation checks.

8.4 Categorical Value Consistency

Categorical columns will be inspected for inconsistent spelling, capitalization, whitespace, and unexpected labels. This includes return status, seasons, days of the week, product categories, payment methods, device types, and membership tiers.

Values will not be changed unless the inspection identifies a clear inconsistency.

In [19]:
```
# Inspect distinct values in categorical columns
categorical_columns = [
    "Returned",
    "OrderSeason",
```

```
    "OrderDayOfWeek",
    "ProductCategory",
    "ProductSubCategory",
    "PaymentMethod",
    "DeviceType",
    "MembershipTier"
]

for column in categorical_columns:
    print(f"\n--- {column} ---")
    print(sorted(df_clean[column].dropna().unique()))
```

```
--- Returned ---
['No', 'Yes']

--- OrderSeason ---
['Fall', 'Spring', 'Summer', 'Winter']

--- OrderDayOfWeek ---
['Friday', 'Monday', 'Saturday', 'Sunday', 'Thursday', 'Tuesday', 'Wednesd
ay']

--- ProductCategory ---
['Automotive', 'Beauty & Personal Care', 'Books', 'Clothing', 'Electronic
s', 'Grocery', 'Health & Wellness', 'Home & Kitchen', 'Sports & Outdoors',
'Toys & Games']

--- ProductSubCategory ---
['Accessories', 'Action Figures', 'Appliances', 'Bakery', 'Bedding', 'Beve
rages', 'Board Games', 'Bottoms', 'Cameras', 'Camping Gear', 'Car Accessor
ies', 'Car Care', 'Cookware', 'Cycling', 'Dairy', 'Decor', 'Educational',
'Electronics', 'Fiction', 'Fitness Accessories', 'Fitness Equipment', 'Foo
twear', 'Fragrances', 'Haircare', 'Headphones', 'History', 'Laptops', 'Mak
eup', 'Medical Devices', 'Mental Wellness', 'Non-Fiction', 'Oral Care', 'O
rganic', 'Outdoor Toys', 'Outerwear', 'Parts', 'Puzzles', 'Science', 'Self
-Help', 'Skincare', 'Smartphones', 'Snacks', 'Storage', 'Supplements', 'Ta
blets', 'Team Sports', 'Tools', 'Tops', 'Vitamins', 'Water Sports']

--- PaymentMethod ---
['Apple Pay', 'Buy Now Pay Later', 'Credit Card', 'Debit Card', 'Gift Car
d', 'Google Pay', 'PayPal']

--- DeviceType ---
['Desktop', 'Mobile', 'Tablet']

--- MembershipTier ---
['Bronze', 'Gold', 'Platinum', 'Silver']
```

Categorical Value Consistency — Initial Findings

The distinct values in the categorical columns were inspected for inconsistent spelling, capitalization, and unexpected labels.

- `Returned` contains `Yes` and `No`.
- `OrderSeason` contains Fall, Spring, Summer, and Winter.
- `OrderDayOfWeek` contains all seven days of the week.
- `ProductCategory`, `ProductSubCategory`, `PaymentMethod`, `DeviceType`, and `MembershipTier` contain clearly formatted category

labels.

Initial cleaning decision: No obvious spelling or capitalization inconsistencies were found in the displayed distinct values. Further checks will be performed for extra whitespace and consistency between the order dates, seasons, and weekdays.

In [21]:
```
# Check text columns for leading or trailing spaces
text_columns = df_clean.select_dtypes(include=["str"]).columns

for column in text_columns:
    has_extra_spaces = df_clean[column].apply(
        lambda value: isinstance(value, str) and value != value.strip()
    )

    if has_extra_spaces.any():
        print(f"{column}: {has_extra_spaces.sum()} values with extra spac

print("Whitespace check completed.")
```

```
Whitespace check completed.
```

Whitespace Validation — Findings

All text columns were checked for leading and trailing whitespace.

- Columns with extra whitespace: None detected.

Cleaning decision: No whitespace removal was required because the check did not identify any values with leading or trailing spaces. The dataset remains unchanged by this check.

In [22]:
```
# Convert OrderDate to datetime for validation
parsed_dates = pd.to_datetime(
    df_clean["OrderDate"],
    format="%m/%d/%Y",
    errors="coerce"
)

# Calculate the expected weekday and season
expected_weekday = parsed_dates.dt.day_name()

month = parsed_dates.dt.month

expected_season = month.map({
    12: "Winter", 1: "Winter", 2: "Winter",
    3: "Spring", 4: "Spring", 5: "Spring",
    6: "Summer", 7: "Summer", 8: "Summer",
    9: "Fall", 10: "Fall", 11: "Fall"
})

# Compare recorded and expected values
weekday_mismatch = df_clean["OrderDayOfWeek"] != expected_weekday
season_mismatch = df_clean["OrderSeason"] != expected_season

print("Weekday mismatches:", weekday_mismatch.sum())
print("Season mismatches:", season_mismatch.sum())
```

```
print("\nWeekday mismatch examples:")
display(df_clean.loc[
    weekday_mismatch,
    ["OrderDate", "OrderDayOfWeek"]
].head(10))

print("\nSeason mismatch examples:")
display(df_clean.loc[
    season_mismatch,
    ["OrderDate", "OrderSeason"]
].head(10))
```

```
Weekday mismatches: 0
Season mismatches: 0

Weekday mismatch examples:
```

OrderDate	OrderDayOfWeek

```
Season mismatch examples:
```

OrderDate	OrderSeason

Date and Category Consistency — Findings

The recorded weekdays and seasons were compared with the values calculated from `OrderDate`.

- Weekday mismatches: 0
- Season mismatches: 0

All recorded weekdays match the actual weekdays calculated from the order dates. All recorded seasons match the defined meteorological seasons.

Cleaning decision: No corrections were required for `OrderDayOfWeek` or `OrderSeason` because all records passed both validation checks.

In [23]:
```
# Inspect distinct values in the remaining text columns
remaining_text_columns = [
    "FirstName",
    "LastName",
    "State",
    "City"
]

for column in remaining_text_columns:
    print(f"\n--- {column} ---")
    print("Number of unique values:", df_clean[column].nunique())
    print("Sample values:")
    print(sorted(df_clean[column].dropna().unique())[:30])
```

```
--- FirstName ---
Number of unique values: 50
Sample values:
['Amanda', 'Andrew', 'Anthony', 'Ashley', 'Barbara', 'Betty', 'Brian', 'Ca
rol', 'Daniel', 'David', 'Donald', 'Donna', 'Dorothy', 'Edward', 'Emily',
'George', 'James', 'Jason', 'Jennifer', 'Jessica', 'John', 'Joseph', 'Josh
ua', 'Karen', 'Kenneth', 'Kevin', 'Kimberly', 'Laura', 'Linda', 'Lisa']

--- LastName ---
Number of unique values: 40
Sample values:
['Adams', 'Allen', 'Anderson', 'Baker', 'Brown', 'Campbell', 'Carter', 'Cl
ark', 'Davis', 'Garcia', 'Green', 'Hall', 'Harris', 'Jackson', 'Johnson',
'Jones', 'King', 'Lee', 'Lewis', 'Martin', 'Martinez', 'Miller', 'Mitchel
l', 'Nelson', 'Perez', 'Phillips', 'Roberts', 'Robinson', 'Sanchez', 'Scot
t']

--- State ---
Number of unique values: 50
Sample values:
['AK', 'AL', 'AR', 'AZ', 'CA', 'CO', 'CT', 'DE', 'FL', 'GA', 'HI', 'IA',
'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA', 'MD', 'ME', 'MI', 'MN', 'MO', 'M
S', 'MT', 'NC', 'ND', 'NE', 'NH']

--- City ---
Number of unique values: 48
Sample values:
['Albany', 'Allentown', 'Atlanta', 'Augusta', 'Aurora', 'Austin', 'Bellevu
e', 'Bristol', 'Buffalo', 'Charlotte', 'Chicago', 'Cincinnati', 'Clevelan
d', 'Clinton', 'Columbus', 'Dallas', 'Durham', 'Erie', 'Fairview', 'George
town', 'Greensboro', 'Greenville', 'Houston', 'Jacksonville', 'Joliet', 'L
os Angeles', 'Macon', 'Madison', 'Miami', 'Naperville']
```

In [24]:

```python
# Check name and location columns for formatting inconsistencies
columns_to_check = ["FirstName", "LastName", "State", "City"]

for column in columns_to_check:
    values = df_clean[column].dropna()

    # Check for leading/trailing spaces
    spaces = values[values != values.str.strip()]

    # Check for values that are not in title case,
    # except State, which should use uppercase codes
    if column == "State":
        inconsistent_case = values[values != values.str.upper()]
    else:
        inconsistent_case = values[values != values.str.title()]

    print(f"\n--- {column} ---")
    print("Values with extra spaces:", len(spaces))
    print("Values with inconsistent capitalization:", len(inconsistent_ca

    if len(spaces) > 0:
        print("Examples with extra spaces:", spaces.unique()[:10])

    if len(inconsistent_case) > 0:
        print("Examples with inconsistent capitalization:",
              inconsistent_case.unique()[:10])
```

```
--- FirstName ---
Values with extra spaces: 0
Values with inconsistent capitalization: 0

--- LastName ---
Values with extra spaces: 0
Values with inconsistent capitalization: 0

--- State ---
Values with extra spaces: 0
Values with inconsistent capitalization: 0

--- City ---
Values with extra spaces: 0
Values with inconsistent capitalization: 0
```

Name and Location Formatting — Findings

The `FirstName`, `LastName`, `State`, and `City` columns were checked for leading/trailing spaces and inconsistent capitalization.

- Extra spaces detected: 0 across all four columns.
- Inconsistent capitalization detected: 0 across all four columns.

State codes use uppercase formatting, while names and city values follow title case.

Cleaning decision: No formatting corrections were required for these columns. The original values were retained.

In [25]:
```
# Inspect column names for formatting issues
print("Current column names:")
print(df_clean.columns.tolist())

print("\nColumn names with leading/trailing spaces:")
print([
    column for column in df_clean.columns
    if column != column.strip()
])

print("\nDuplicate column names:", df_clean.columns.duplicated().sum())
```

```
Current column names:
['OrderID', 'CustomerID', 'FirstName', 'LastName', 'State', 'City', 'Order
Date', 'OrderSeason', 'OrderDayOfWeek', 'ProductCategory', 'ProductSubCate
gory', 'UnitPrice', 'Quantity', 'TotalAmount', 'DiscountPercent', 'Discoun
tAmount', 'FinalAmount', 'PaymentMethod', 'DeviceType', 'MembershipTier',
'ShippingDays', 'Returned', 'ReviewScore']

Column names with leading/trailing spaces:
[]

Duplicate column names: 0
```

Column Name Validation — Findings

The dataset's column names were checked for leading/trailing spaces and duplicate names.

- Total columns: 23
- Column names with leading/trailing spaces: 0
- Duplicate column names: 0

Cleaning decision: No column renaming was required because all column names passed the validation checks. The existing column names were retained.

In [26]:
```
# Convert OrderDate from text to datetime
df_clean["OrderDate"] = pd.to_datetime(
    df_clean["OrderDate"],
    format="%m/%d/%Y"
)

# Verify the conversion
print("OrderDate data type:", df_clean["OrderDate"].dtype)
print("Missing dates after conversion:", df_clean["OrderDate"].isna().sum

display(df_clean[["OrderID", "OrderDate"]].head())
```

```
OrderDate data type: datetime64[us]
Missing dates after conversion: 0
```

	OrderID	OrderDate
0	ORD-010001	2022-05-22
1	ORD-010002	2024-10-17
2	ORD-010003	2024-06-08
3	ORD-010004	2024-10-24
4	ORD-010005	2023-07-22

Date Conversion — Results

The `OrderDate` column was converted from text to a datetime data type.

- Data type after conversion: `datetime64[us]`
- Missing dates after conversion: 0
- Date conversion errors: 0

Cleaning decision: The conversion was successful, and all dates remained valid. The datetime column will be retained in the cleaned dataset to support sorting and date-based analysis.

In [27]:
```
# Final verification of the cleaned dataset
print("Final number of rows:", len(df_clean))
print("Number of columns:", df_clean.shape[1])
print("Missing-value cells:", df_clean.isna().sum().sum())
print("Exact duplicate rows:", df_clean.duplicated().sum())
print("OrderDate data type:", df_clean["OrderDate"].dtype)

print("\nFinal dataset information:")
df_clean.info()
```

```
Final number of rows: 1980
Number of columns: 23
Missing-value cells: 0
Exact duplicate rows: 0
OrderDate data type: datetime64[us]

Final dataset information:
<class 'pandas.DataFrame'>
Index: 1980 entries, 0 to 1998
Data columns (total 23 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   OrderID             1980 non-null   str
 1   CustomerID          1980 non-null   str
 2   FirstName           1980 non-null   str
 3   LastName            1980 non-null   str
 4   State               1980 non-null   str
 5   City                1980 non-null   str
 6   OrderDate           1980 non-null   datetime64[us]
 7   OrderSeason         1980 non-null   str
 8   OrderDayOfWeek      1980 non-null   str
 9   ProductCategory     1980 non-null   str
 10  ProductSubCategory  1980 non-null   str
 11  UnitPrice           1980 non-null   float64
 12  Quantity            1980 non-null   int64
 13  TotalAmount         1980 non-null   float64
 14  DiscountPercent     1980 non-null   int64
 15  DiscountAmount      1980 non-null   float64
 16  FinalAmount         1980 non-null   float64
 17  PaymentMethod       1980 non-null   str
 18  DeviceType          1980 non-null   str
 19  MembershipTier      1980 non-null   str
 20  ShippingDays        1980 non-null   int64
 21  Returned            1980 non-null   str
 22  ReviewScore         1980 non-null   int64
dtypes: datetime64[us](1), float64(4), int64(4), str(14)
memory usage: 371.2 KB
```

In [28]:
```python
# Save the cleaned dataset
output_path = "../data/cleaned/ecommerce_cleaned.csv"

df_clean.to_csv(output_path, index=False)

print("Cleaned dataset saved successfully!")
print("File path:", output_path)
```

```
Cleaned dataset saved successfully!
File path: ../data/cleaned/ecommerce_cleaned.csv
```

In [29]:
```python
# Reload the saved CSV and verify its contents
df_saved = pd.read_csv(output_path)

print("Saved dataset rows:", df_saved.shape[0])
print("Saved dataset columns:", df_saved.shape[1])
print("Missing-value cells:", df_saved.isna().sum().sum())
print("Exact duplicate rows:", df_saved.duplicated().sum())
```

```
Saved dataset rows: 1980
Saved dataset columns: 23
Missing-value cells: 0
Exact duplicate rows: 0
```

Final Cleaning Summary

The e-commerce dataset was inspected and cleaned according to the Week 1 data-cleaning requirements.

Final Dataset Statistics

- Original records: 2,000
- Records after removing exact duplicate rows: 1,980
- Columns retained: 23
- Missing-value cells: 0
- Exact duplicate rows remaining: 0

Cleaning Decisions

1. **Missing values:** No imputation or row removal was needed because no missing values were detected.
2. **Duplicate records:** Removed 20 exact duplicate rows while preserving one copy of each record.
3. **Repeated order IDs:** Investigated separately; the repeated IDs had identical full records, so the extra copies were treated as duplicates.
4. **Date validation:** All dates were valid and fell between January 1, 2022, and February 28, 2025.
5. **Date conversion:** Converted `OrderDate` from text to datetime.
6. **Transaction amounts:** All tested total, discount, and final amounts matched their calculated values.
7. **Numeric validation:** All records passed the quantity, unit-price, discount percentage, shipping duration, and review-score checks.
8. **Categorical consistency:** No weekday or season mismatches were found.
9. **Text formatting:** No leading/trailing spaces or capitalization inconsistencies were detected in the inspected columns.
10. **Column names:** No leading/trailing spaces or duplicate column names were found.

Output

The cleaned dataset was saved separately as `data/cleaned/ecommerce_cleaned.csv`. The original raw dataset was preserved.

The cleaned dataset is ready for subsequent exploratory data analysis.